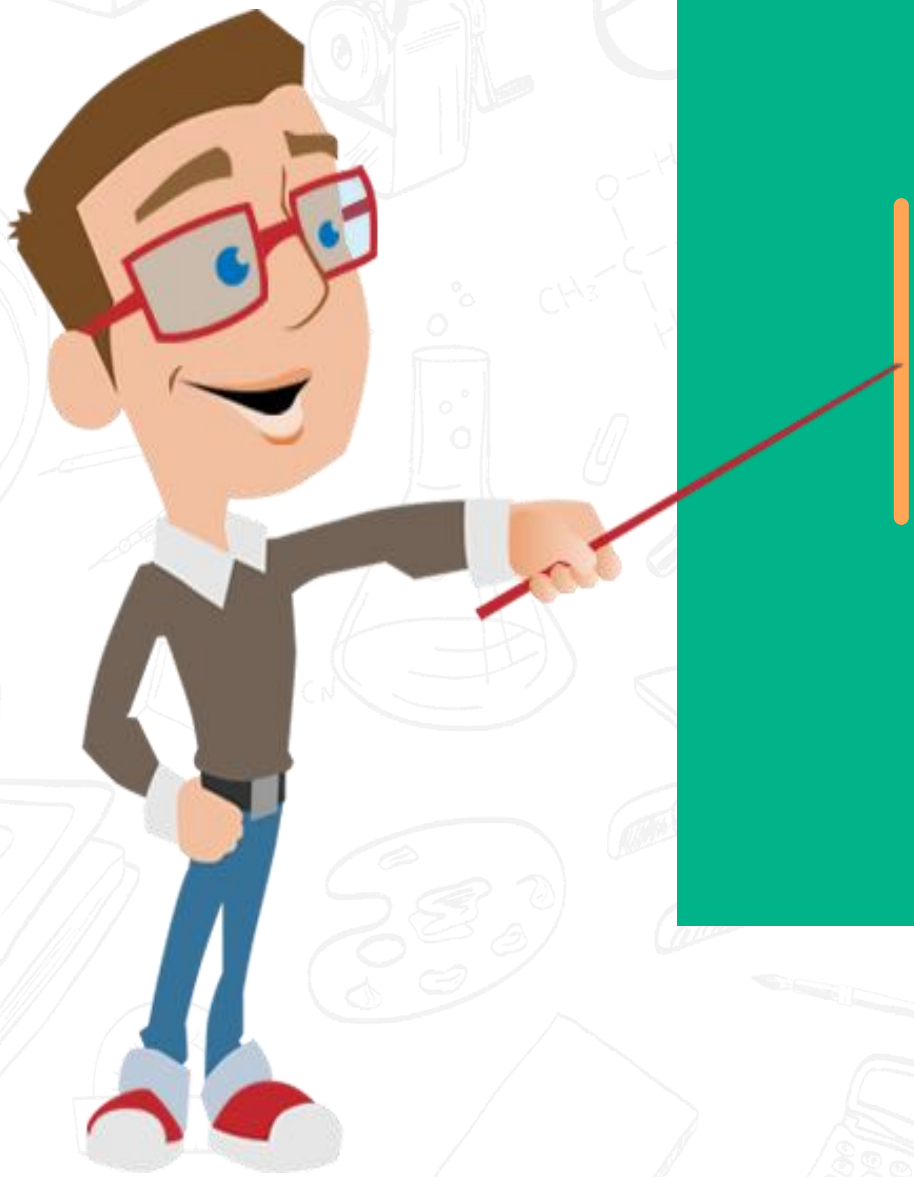


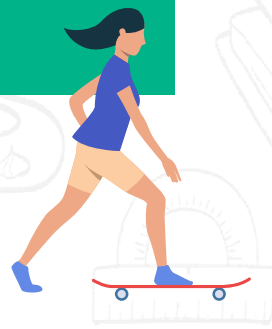
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 6-1

1 ROS Filesystem(1)

2 ~

3 ~

4 ~

Objectives

1、了解ROS系统的三层架构

2、掌握ROS文件系统架构

3、掌握创建工作空间和功能包的方法



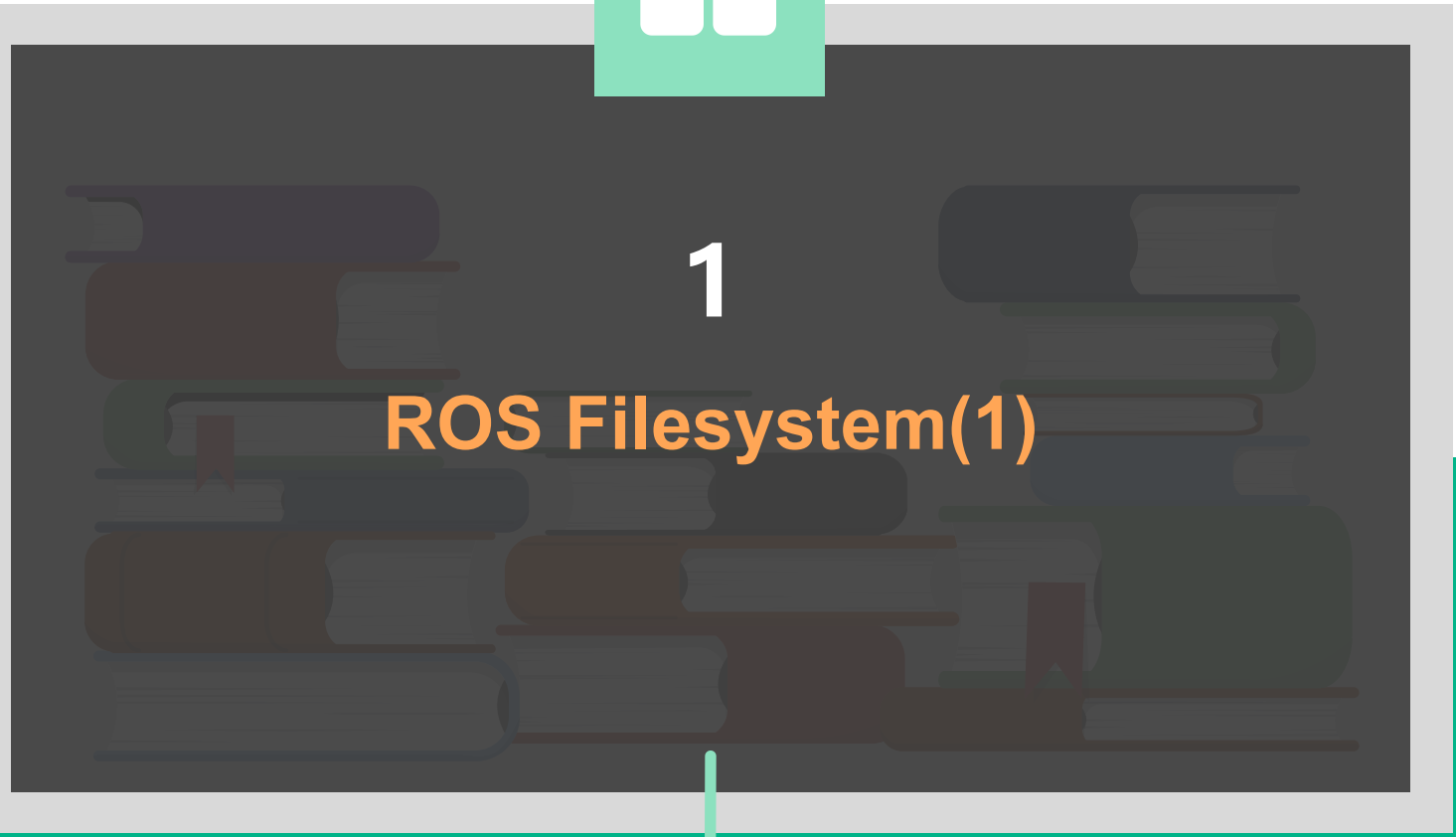
Pre-assessment

1、ROS文件系统指什么?

2、是否使用过catkin_make?

3、是否了解工作空间？工作空间的用处?

4、是否自己创建过catkin功能包?



1

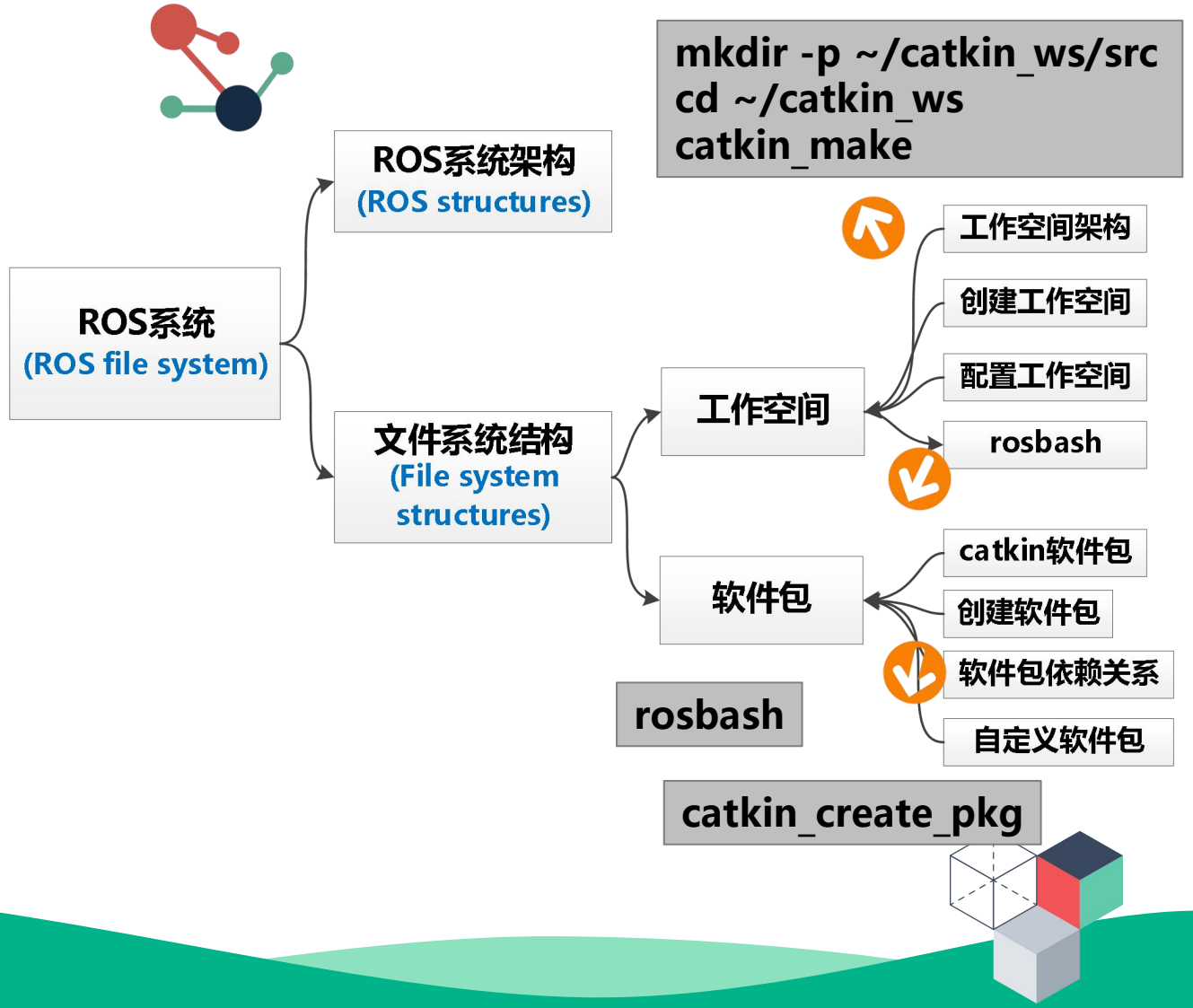
ROS Filesystem(1)

1.1 Knowledge architecture

1.1.1 Overview

- **ROS系统架构**：ROS系统的架构包含三个层次：系统层、中间层、应用层

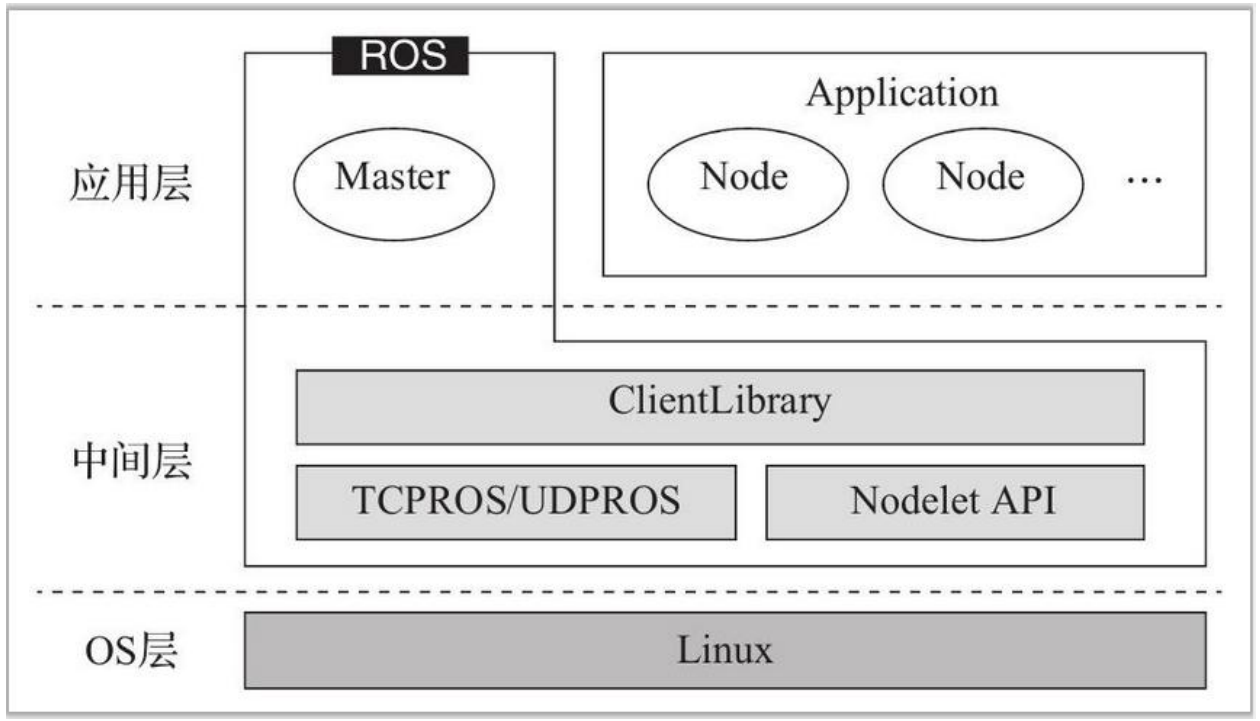
- **文件系统结构**：
  - **工作空间**：
    - 工作空间结构
    - 创建并配置工作空间
  - **功能包**：
    - catkin功能包定义
    - 创建功能包，功能包依赖关系
    - 自定义功能包



1.2 ROS architecture

1.2.1 ROS system architecture

➢ **ROS系统架构主要被设计为三个层次：**



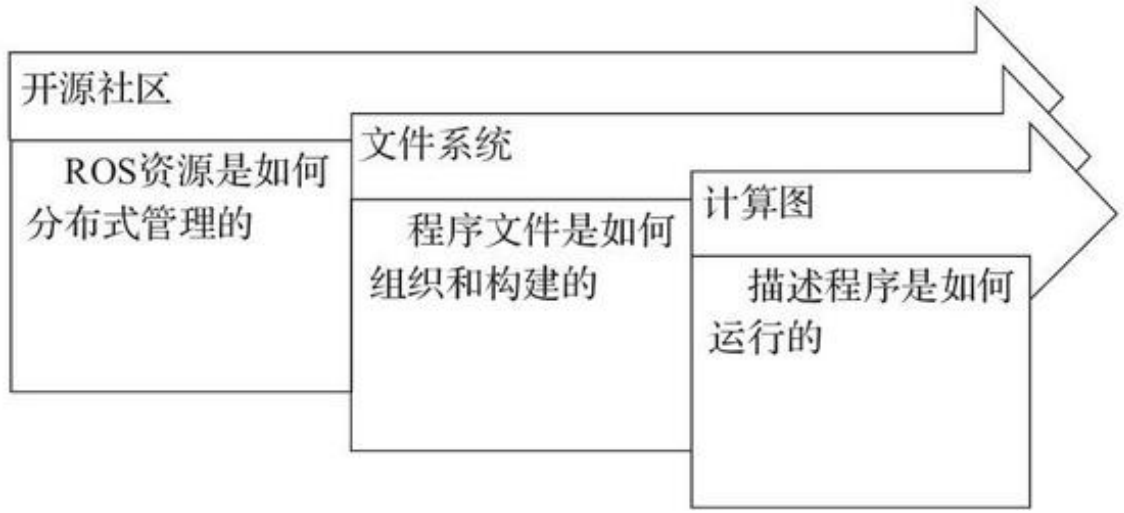
- **系统层：**
  - **Linux(Ubuntu)**
- **中间层：通信系统**
  - **发布/订阅**
  - **客户端/服务器端**
  - **Nodelet**
- **应用层：ROS应用**
  - **节点**
  - **话题**
  - **功能包**

1.2 ROS architecture

1.2.2 ROS own architecture

http://wiki.ros.org/ROS/Concepts

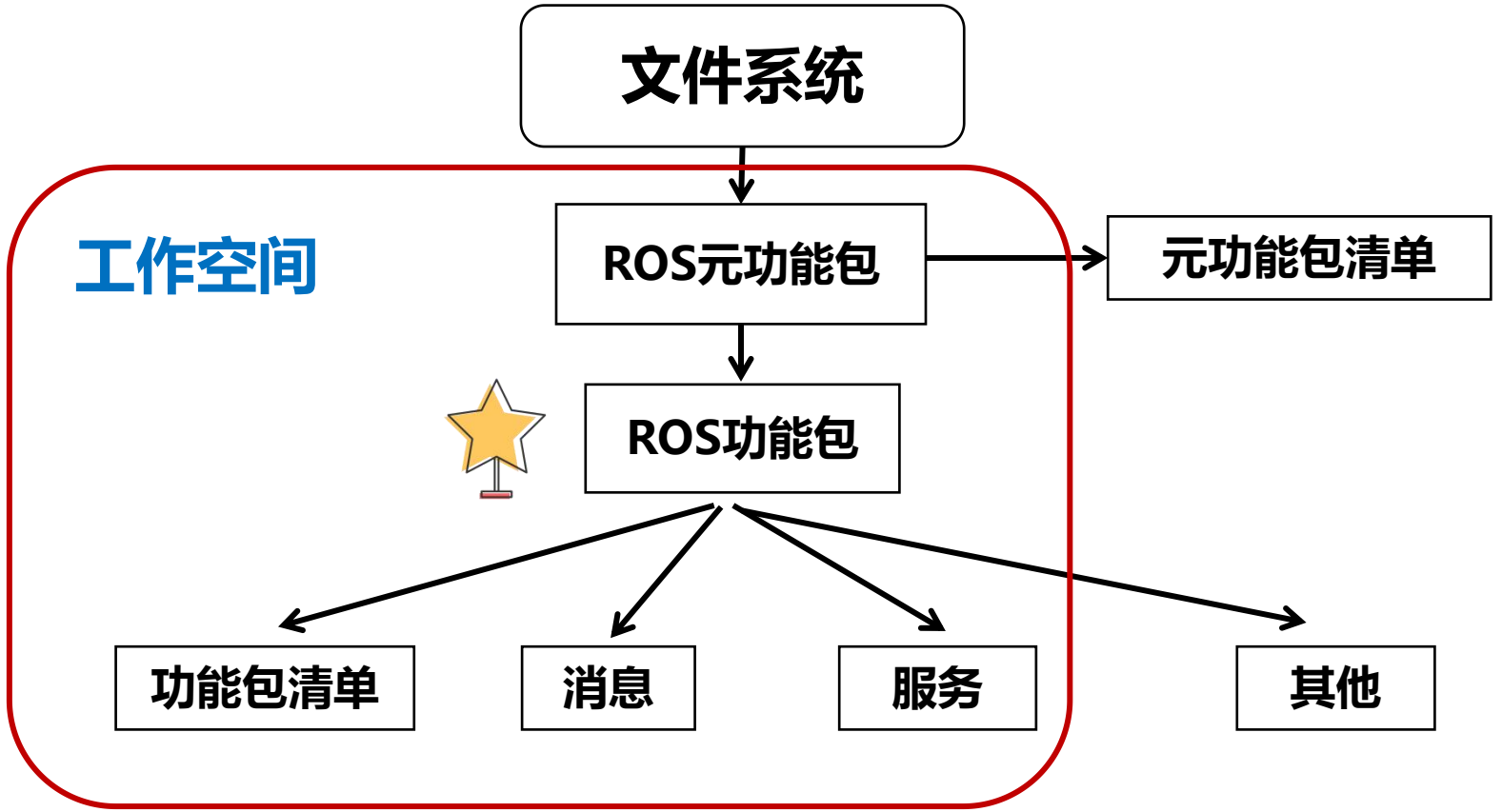
➢ **ROS自身系统实现的三个层次：**



- **文件系统级：** ROS将所有的文件按照一定的规则进行组织，不同功能的文件被放置在不同的文件夹
- **计算图级：** ROS 处理数据的一种点对点的网络形式。ROS 中的基本计算图级概念包括：节点、节点管理器、参数服务器、消息、服务、话题和功能包
- **开源社区级：** 丰富的社区资源，发行版，软件源，ROS wiki，ROR Answers

1.3 The filesystem level

1.3.1 Overview ROS filesystem: http://wiki.ros.org/ROS/Concepts



1.3 The filesystem level

1.3.2 ROS workspace structures

- 工作空间是一个包含功能包、可编辑源文件或编译包的文件夹，其包含 build、devel、src 子文件夹

a) src 源文件空间(The source space)：

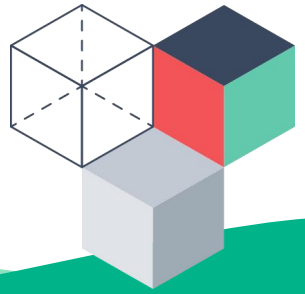
- 创建工作空间时同时创建src源文件空间，包含 CMakeLists.txt（负责调用CMake编译）
- 在源文件空间中放置了功能包、项目、克隆包等

b) build 编译空间(The build space)：

- 在build文件夹中，CMake 和 catkin 为功能包和项目保存缓存信息、配置和其他中间文件

c) devel 开发空间(The development space)：

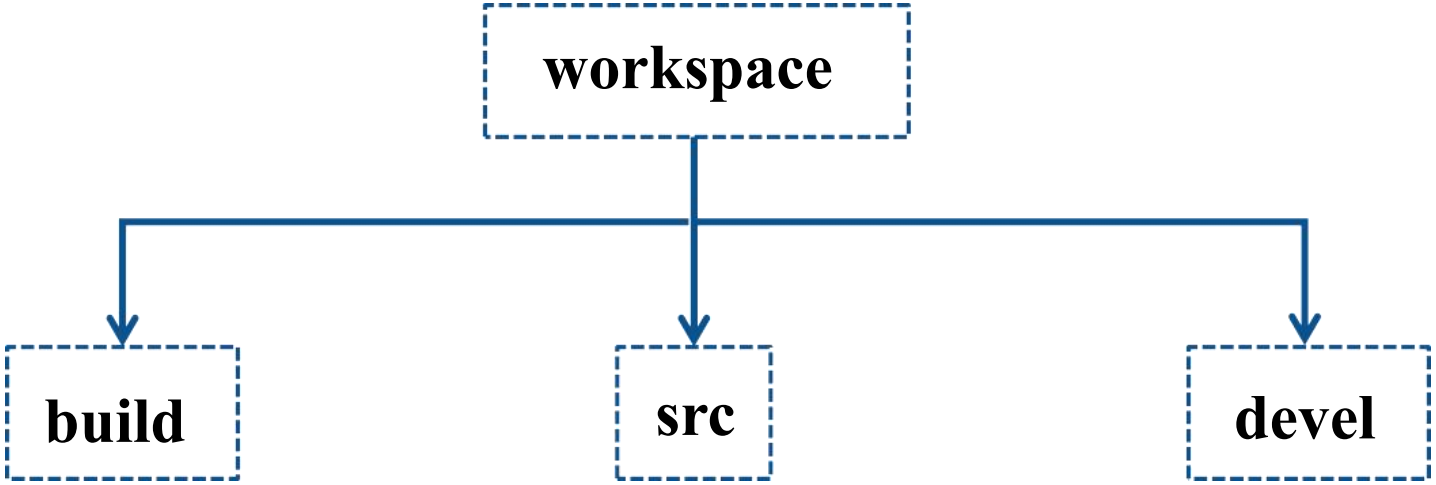
- devel文件夹用来保存编译后的程序



1.3 The filesystem level

1.3.2 ROS workspace structures

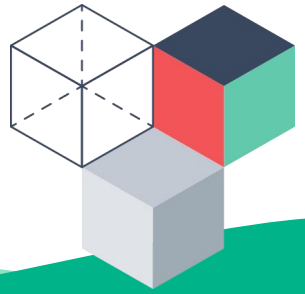
workspace：存放工程文件的文件夹



编译过程产生的中间文件

代码文件

编译生成的目标文件



1.3 The filesystem level

1.3.3 Command-line tools for ROS workspace

1. 创建工作空间：

```
$ mkdir -p ~/catkin_ws/src

$ cd ~/catkin_ws  # 编译需要在 catkin_ws 中进行

$ catkin_make     # 编译工作空间
```

```
ros@ROS-VirtualBox:~/catkin_ws$ ls
build  devel  src
```

```
ros@ROS-VirtualBox:~/catkin_ws$ catkin_make
Base path: /home/ros/catkin_ws
Source space: /home/ros/catkin_ws/src
Build space: /home/ros/catkin_ws/build
Devel space: /home/ros/catkin_ws/devel
Install space: /home/ros/catkin_ws/install
Creating symlink "/home/ros/catkin_ws/src/CMakeLists.txt" pointing to "/opt/ros/melo
####
#### Running command: "cmake /home/ros/catkin_ws/src -DCATKIN_DEVEL_PREFIX=/home/ros
 Unix Makefiles" in "/home/ros/catkin_ws/build"
####
-- The C compiler identification is GNU 7.5.0
-- The CXX compiler identification is GNU 7.5.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Using CATKIN_DEVEL_PREFIX: /home/ros/catkin_ws/devel
-- Using CMAKE_PREFIX_PATH: /opt/ros/melodic
-- This workspace overlays: /opt/ros/melodic
-- Found PythonInterp: /usr/bin/python2 (found suitable version "2.7.17", minimum re
-- Using PYTHON_EXECUTABLE: /usr/bin/python2
-- Using Debian Python package layout
-- Using empy: /usr/bin/empy
-- Using CATKIN_ENABLE_TESTING: ON
-- Call enable_testing()
-- Using CATKIN_TEST_RESULTS_DIR: /home/ros/catkin_ws/build/test_results
-- Found gtest sources under '/usr/src/googletest': gtests will be built
-- Found gmock sources under '/usr/src/googletest': gmock will be built
-- Found PythonInterp: /usr/bin/python2 (found version "2.7.17")
```

1.3 The filesystem level

1.3.3 Command-line tools for ROS workspace

2. 配置工作空间（环境变量）：

```
$ echo $ROS_PACKAGE_PATH
```

查看ROS正在使用的工作空间

```
ros@ROS-VirtualBox:~$ echo $ROS_PACKAGE_PATH
/opt/ros/melodic/share
```

```
$ source devel/setup.bash
```

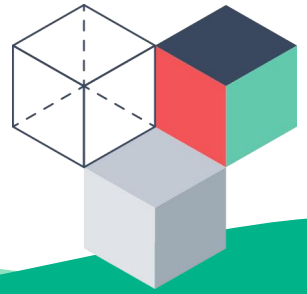
从ROS工作空间重新加载 setup.bash，并覆盖默认配置

```
ros@ROS-VirtualBox:~/catkin_ws$ source devel/setup.bash
ros@ROS-VirtualBox:~/catkin_ws$ echo $ROS_PACKAGE_PATH
/home/ros/catkin_ws/src:/opt/ros/melodic/share
```

```
$ echo “source /opt/ros/noetic/setup.bash” >>~/.bashrc
```

```
$ echo “source /home/your_name/catkin_ws/devel/setup.bash” >>~/.bashrc
```

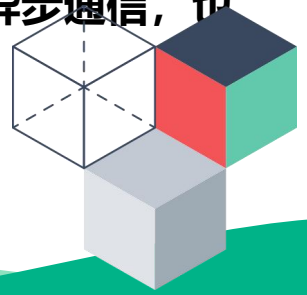
配置 bashrc 文件，使 ROS 和创建的工作空间永久生效



1.3 The filesystem level

1.3.4 ROS package

- **功能包(package)：实现特定功能的软件包，功能包中包括：功能包清单、消息、服务、代码、其他**
  - 功能包是 ROS 中整理和组织软件的主要单元，也是 ROS 中能够建立和分享的最小单元
  - 功能包中包含ROS节点、ROS程序库、数据集、配置文件、可执行文件等
  - **功能包清单(package manifest)：**
    - 功能包清单是一个名为 package.xml 的 XML 文件，其必须位于功能包文件夹中
    - 功能包清单中列举了功能包的相关信息，包括功能包名称、版本、作者、开源许可、依赖关系等
  - **消息类型(message type)：**
    - 消息是指，一个 ROS 节点发送到其他 ROS 节点的信息，节点间通过将消息发布到话题来实现异步通信，也指 ROS 服务中的请求和响应消息
    - 消息位于所在功能包下的 msg 文件夹中，扩展名为.msg，



1.3 The filesystem level

1.3.4 ROS package

- 服务类型(service type)：
  - 定义了 ROS 中由每个 ROS 节点提供的关于服务请求和响应的数据结构
  - ROS节点通过调用系统服务来同步交换请求和响应消息
  - 请求和响应消息位于功能包下的 srv 文件中，扩展名为.srv

➢ 元功能包(matapackage)：

- Metapackages 是一组具体的服务相关的功能包。例如，将多个功能包组合在一起，以实现特定的目的和功能（例如导航任务）

➢ 元功能包清单(metapackage manifest)：

- 作用类似 package manifest 文件，但是它针对的是 metapackage

turtlesim功能包结构

```
ros@ROS-VirtualBox:/opt/ros/melodic/share/turtlesim$ tree
.
├── cmake
│   ├── turtlesimConfig.cmake
│   ├── turtlesimConfig-version.cmake
│   ├── turtlesim-msg-extras.cmake
│   └── turtlesim-msg-paths.cmake
├── images
│   ├── box-turtle.png
│   ├── diamondback.png
│   ├── electric.png
│   ├── fuerte.png
│   ├── groovy.png
│   ├── hydro.png
│   ├── hydro.svg
│   ├── indigo.png
│   ├── indigo.svg
│   ├── jade.png
│   ├── kinetic.png
│   ├── kinetic.svg
│   ├── lunar.png
│   ├── lunar.svg
│   ├── melodic.png
│   ├── palette.png
│   ├── robot-turtle.png
│   ├── sea-turtle.png
│   └── turtle.png
├── msg
│   ├── Color.msg
│   └── Pose.msg
├── package.xml
└── srv
    ├── Kill.srv
    ├── SetPen.srv
    ├── Spawn.srv
    ├── TeleportAbsolute.srv
    └── TeleportRelative.srv
```

1.3 The filesystem level

1.3.5 Command-line tools for ROS package

1. 创建ROS功能包：

```
$ cd ~/catkin_ws/src
```

```
$ catkin_create_pkg beginner_tutorials std_msgs roscpp rospy
```

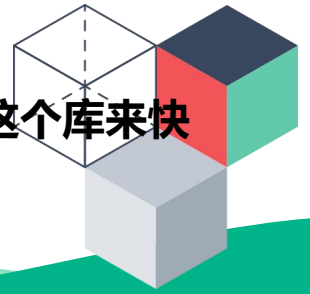
```
$ cd ~/catkin_ws
```

```
$ catkin_make
```

备注：catkin_create_pkg语法格式如下：

```
catkin_create_pkg package_name [dependency1] [dependency2] ... [dependencyN]
```

- 常用 dependencies：
  - std_msgs：标准 ROS 消息，包括表示原始数据类型和其他基本消息结构的常见消息类型
  - roscpp：roscpp 功能包是 ROS 的一个 C++ 客户端库，它提供了一个客户端库，可以通过这个库来快速和 ROS 话题、服务和参数的接口进行交互
  - rospy：rospy是 ROS 的 Python 客户端库



1.3 The filesystem level

1.3.5 Command-line tools for ROS package

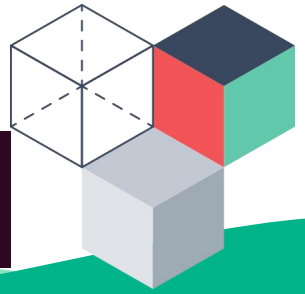
- 适用于 ROS 功能包的命令工具：rospack (ros+package)

  语法格式：rospack command [options] [package]

  - rospack list: 查看软件包列表
  - rospack find package_name： 找到 package 的路径
  - rospack depends1 package_name：列出 package 的一级依赖
  - rospack depends package_name：列出 package 的所有依赖
  - rospack profile：提供新添加到 ROS 中功能包的信息，在安装新功能包时十分有用

  举例：

```
ros@ROS-VirtualBox:~/catkin_ws$ rospack find beginner_tutorials
/home/ros/catkin_ws/src/beginner_tutorials
```



1.3 The filesystem level

1.3.5 Command-line tools for ROS package

2. 软件依赖包关系：一级依赖

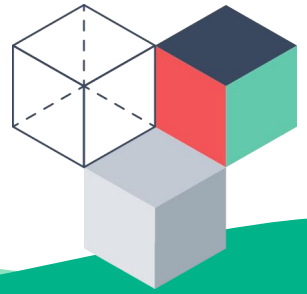
- 使用 catkin_create_pkg 命令时提供了几个功能包作为依赖关系，可以使用 rospack 命令工具来查看这些依赖包

```
$ rospack depends1 beginner_tutorials
```

```
ros@ROS-VirtualBox:~/catkin_ws/devel$ rospack depends1 beginner_tutorials
roscpp
rospy
std_msgs
```

- rospack 列出了在运行 catkin_create_pkg 命令时作为参数的依赖包，这些依赖关系存储在 package.xml 文件中

```
$ roscd beginner_tutorials
```



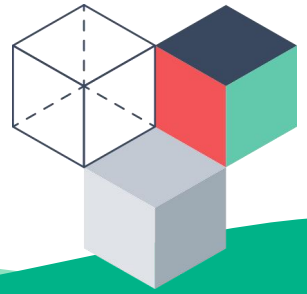
1.3 The filesystem level

1.3.5 Command-line tools for ROS package

2. 软件依赖包关系：一级依赖

```
$ cat package.xml
```

```
<package>
...
  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>
...
</package>
```



1.3 The filesystem level

1.3.5 Command-line tools for ROS package

2. 软件依赖包关系：间接依赖

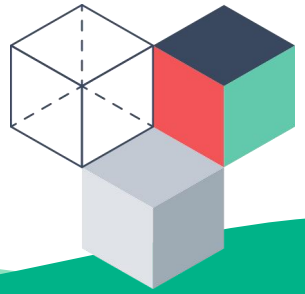
- 很多情况下，一个依赖包还会有自己的依赖关系，比如rospy就有其它依赖包

```
$ rospack depends1 rospy
```

```
ros@ROS-VirtualBox:~/catkin_ws/src$ rospack depends1 rospy
genpy
roscpp
rosgraph
rosgraph_msgs
roslib
std_msgs
```

- 使用rospack可以递归检测出所有嵌套的依赖包

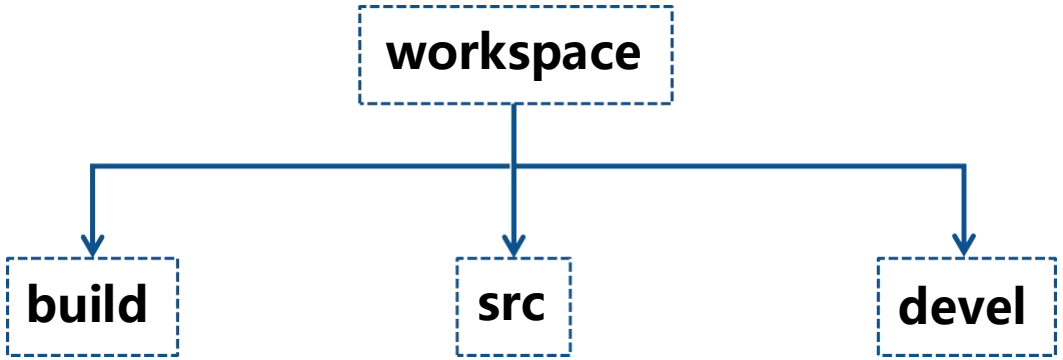
```
$ rospack depends beginner_tutorials
```



1.3 The filesystem level

1.3.6 ROS package review

workspace：存放工程文件的文件夹

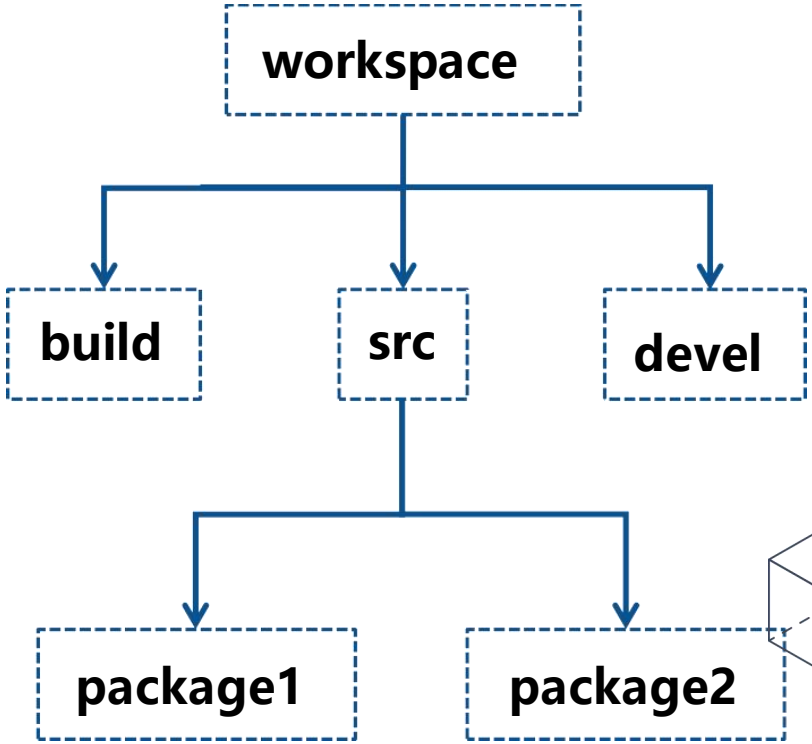


编译过程产生的中间文件

代码文件

编译生成的目标文件

package：存放用户功能代码的文件夹



1.3 The filesystem level

1.3.7 Turtlesim: http://wiki.ros.org/turtlesim

ROS.org About | Support | Discussion Forum | Service Status | Q&A answers.ros.org

Documentation Browse Software News

turtlesim

kinetic melodic noetic *Show EOL distros:* ☐

Documentation Status

***ros_tutorials**: roscpp_tutorials | rospy_tutorials | turtlesim*

Package Summary

✔ Released ✔ Continuous Integration: 24 / 24 ✔ Documented

turtlesim is a tool made for teaching ROS and ROS packages.

- Maintainer status: maintained
- Maintainer: Mabel Zhang <mabel AT openrobotics DOT org>, Shane Loretz <sloretz AT openrobotics DOT org>
- Author: Josh Faust, Dirk Thomas <dthomas AT osrfoundation DOT org>
- License: BSD
- Bug / feature tracker: https://github.com/ros/ros_tutorials/issues
- Source: git https://github.com/ros/ros_tutorials.git (branch: noetic-devel)

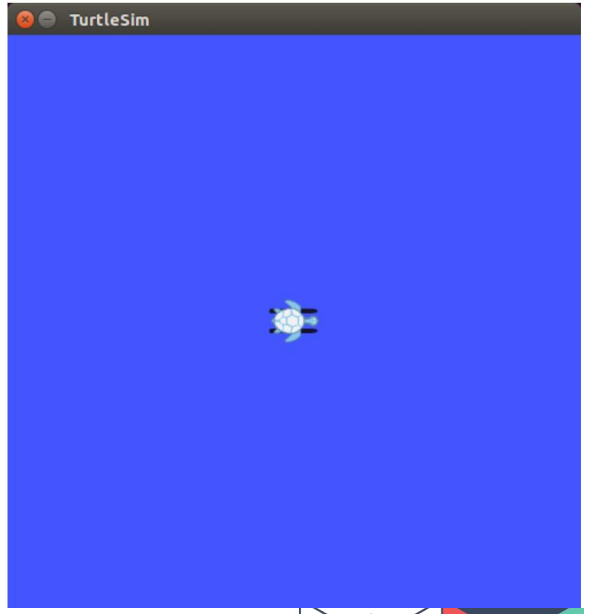
Package Links
- Code API
- Msg/Srv API
- Tutorials
- FAQ
- Changelog
- Change List
- Reviews

Dependencies (11)
Used by (4)
Jenkins jobs (10)

目录
1. Getting Started with Turtlesim
2. Nodes
   1. turtlesim_node
      1. Subscribed Topics
      2. Published Topics
      3. Services
      4. Parameters
   2. mimic
      1. Subscribed Topics
      2. Published Topics
3. Turtlesim Video Tutorials
   1. Python Tutorials

Turtlesim: turtlesim 是用于教授 ROS 和 ROS 功能包的工具

目录
1. Getting Started with Turtlesim
2. Nodes
   1. turtlesim_node
      1. Subscribed Topics
      2. Published Topics
      3. Services
      4. Parameters
   2. mimic
      1. Subscribed Topics
      2. Published Topics
3. Turtlesim Video Tutorials
   1. Python Tutorials
   2. C++ Tutorials



Post-assessment

1. ROS系统架构的三个层次？ROS自身结构的三个层次?
2. ROS工作空间的包含的3个文件夹？分别有何作用?
3. 创建工作空间的3条命令?
4. 创建包含std_msgs、rospy、roscpp的功能包a的命令?

主观题 10分

设置

1. ROS系统架构的三个层次？ROS自身结构的三个层次?
2. ROS工作空间的包含的3个文件夹？分别有何作用?
3. 创建工作空间的3条命令?
4. 创建包含std_msgs、rospy、roscpp的功能包a的命令?

正常使用主观题需2.0以上版本雨课堂

作答

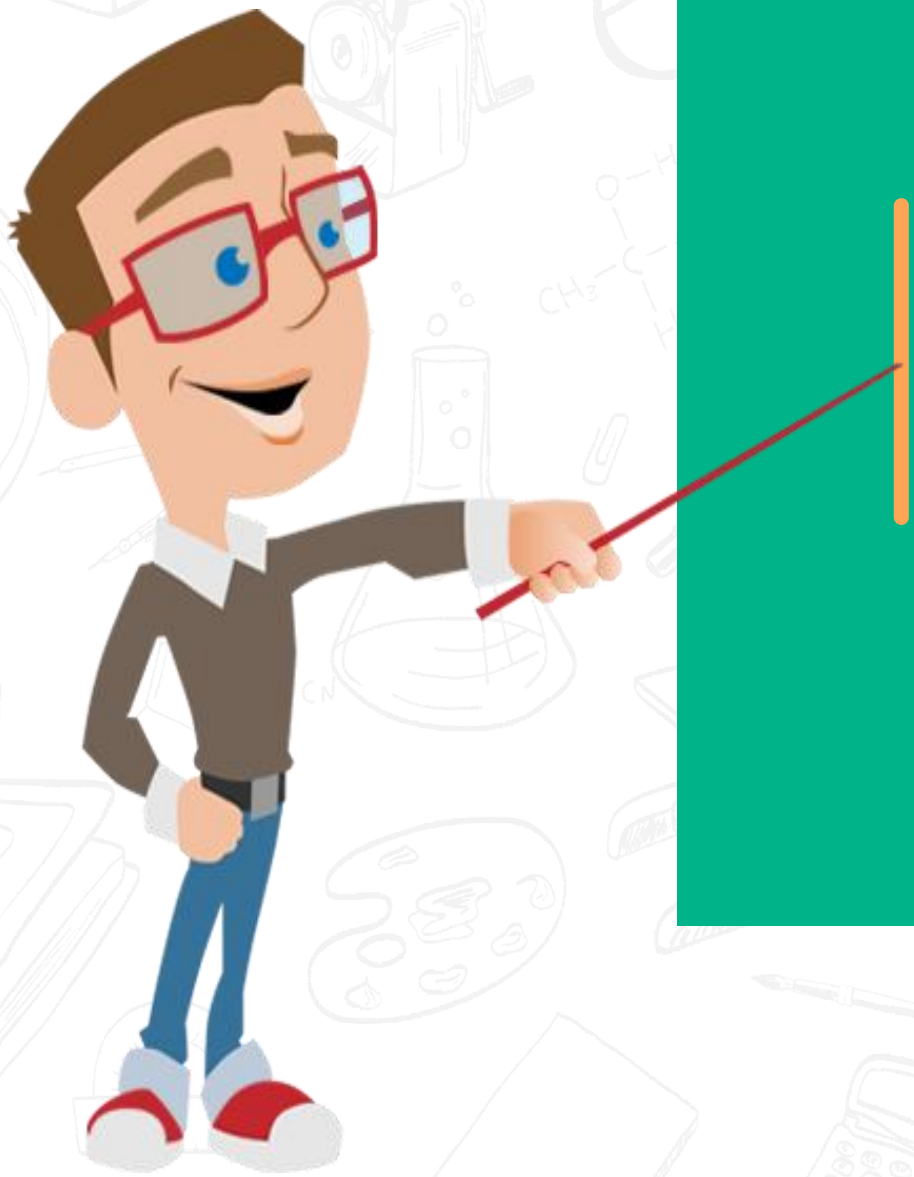
ROS Summary

1. ROS系统架构的三个层次：
    a) 系统层、中间层、应用层
2. ROS系统实现的三层架构：
    a) 文件系统级、计算图级、开源社区级
3. ROS文件系统
    a) 文件系统架构
    b) 工作空间架构 
    c) ROS功能包 

Q&A

今日事今日毕

Question?



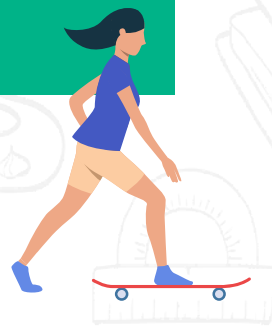
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 6-2

1 XML File

2 ROS Filesystem(2)

3 ~

4 ~

Objectives

1、理解 XML 文件的结构

2、掌握 Catkin 功能包的组织架构

3、掌握 package.xml 的使用方法



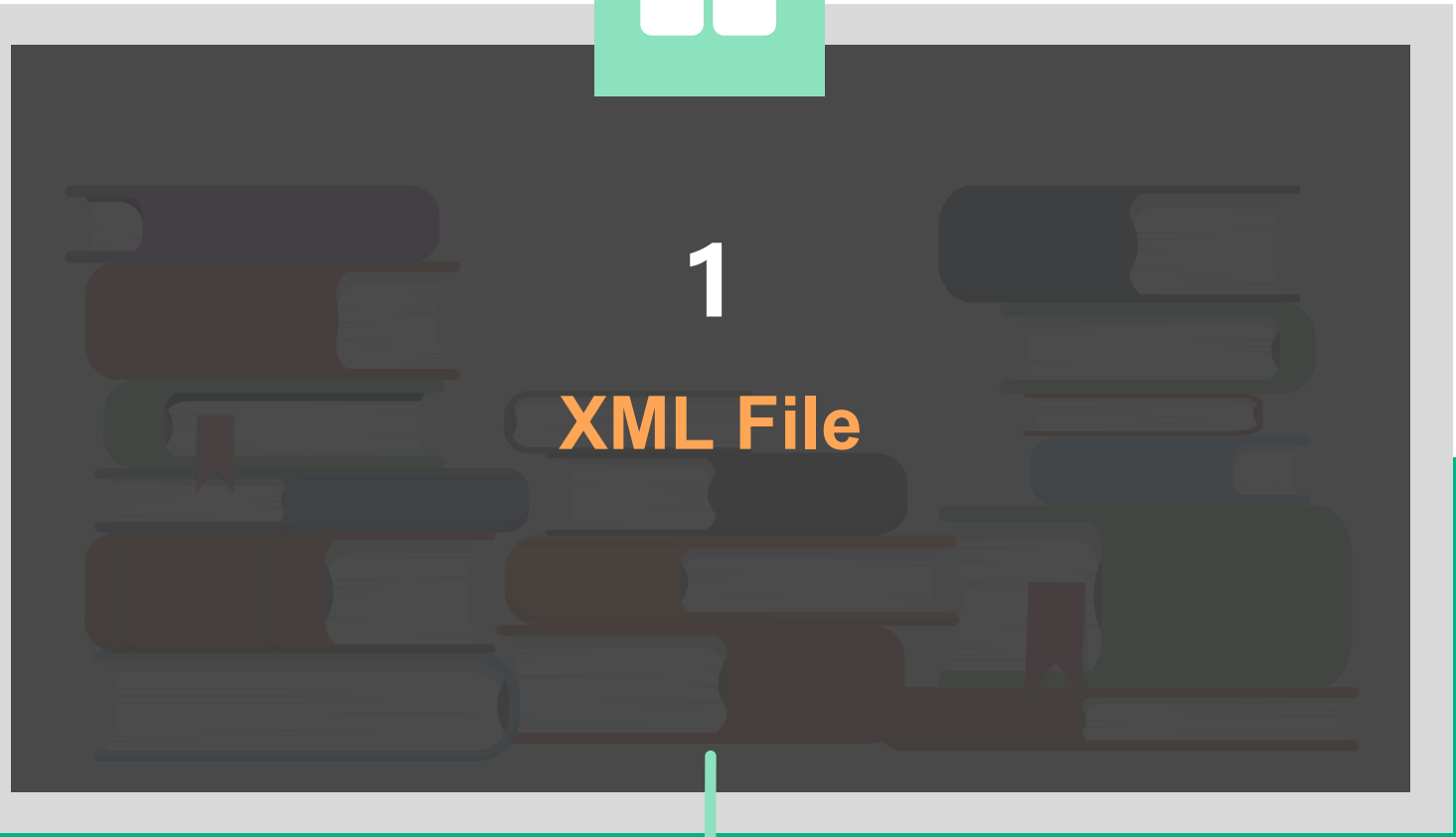
Pre-assessment

1、了解和使用过 XML 文件?

2、是否了解 Cmake?

3、是否了解 Catkin 功能包

4、是否自己创建过 Catkin 功能包?



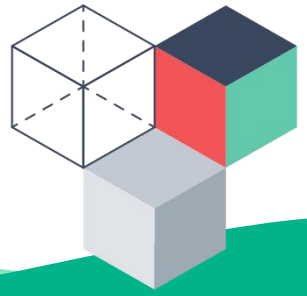
1

XML File

1.1 XML basics

1.1.1 Overview

- XML(Extensible Markup Language)：可扩展标记语言，标记指计算机所能理解的信息符号，通过此种标记，计算机之间可以处理各种信息
  - 特点：XML 与操作系统、编程语言和开发平台无关，可以跨平台使用
  - 作用：配置应用程序（配置文件一般用 XML 编写），实现不用系统间的数据交互、数据共享等
- XML文件结构：
  1. 文档声明
  2. 元素
  3. 属性
  4. 注释
  5. CDATA区 、特殊字符
  6. 处理指令（processing instruction）



1.1.2 XML file structure

1、文档声明：在编写 XML 文档时，需要先使用文档声明，声明 XML 文档的类型。XML 声明是 XML 文档的第一句，其格式如下：

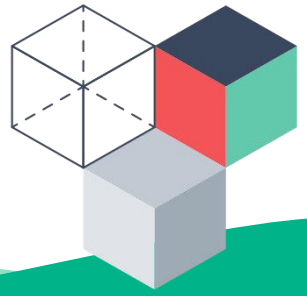
```
<?xml version="1.0" encoding="utf-8"?>
```

- **声明包括两个部分：① XML版本；② 文字编码方式**

2、元素：指 XML 文件中出现的标签，一个标签分为 开始标签 和 结束标签，使用标签时需注意：

- **标签必须成对出现，开始标签+结束标签**





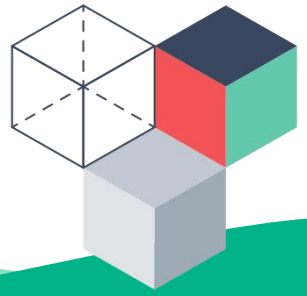
1.1 XML basics

1.1.2 XML file structure

- 标签内容对大小写敏感
- 标签可以嵌套，但不可以交叉嵌套

```
<title><name>XML编程</title></name>
<title><name>XML编程</name></title>
```

- 声明之后第一个元素为根元素，XML 文档有且只有一个根元素：
  - 其他元素都是这个根元素的子元素，根元素完全包括文档中其他所有的元素
  - 根元素的起始标记要放在所有其他元素的起始标记之前
  - 根元素的结束标记要放在所有其他元素的结束标记之后



1.1 XML basics

1.1.2 XML file structure



3、属性：一个标签可以有多个属性，每个属性都有它自己的 名称 和 取值

- 属性语法：<元素名 属性名=“属性值”/>，举例：

```
<Student ID=“S100”>
    <Name>Tom</Name>
</Student>
```

1.1.2 XML file structure

- **注意**：
  - 属性值要用双引号（"）或单引号（'）引起来
  - 定义属性必须遵循与标签相同的**命名规范**，即
    - 名称中可以包含字母、数字或者其他的字符
    - 名称不能以数字或者标点符号开头
    - 名称中不能包含空格
    - 名称中间不能包含冒号
- 一个元素可以有多个属性，它的基本格式为：<元素名 属性名=“属性值” 属性名=“属性值”>

4、注释：XML文件中的注释语法：

<!--注释-->





2

ROS Filesystem(2)

2.1 The filesystem level

2.1.1 Catkin package

- **一个功能包要想称为 catkin package，必须符合以下要求：**

  a) **这个功能包必须有一个符合 catkin 规范的 package.xml 文件**

  - **package.xml 文件提供有关该功能包的基础信息**

  b) **这个包必须有一个 catkin 版本的 CMakeLists.txt 文件**

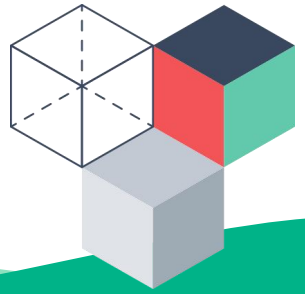
  - **CMakeLists.txt 文件保存 Cmake 的配置信息，如果是 Catkin 元功能包的话，则需要有一个 CMakeList.txt 文件的相关样板**

  c) **每个包必须有自己的目录**

  - **在同一个目录下不能有嵌套的或者多个功能包存在**

- **最简单的 catkin 功能包示例：**

```
my_package/
  CMakeLists.txt
  package.xml
```

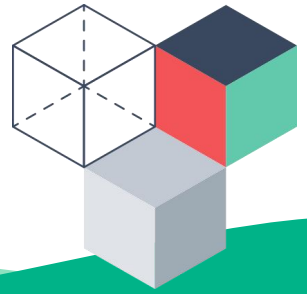


2.1 The filesystem level

2.1.1 Catkin package

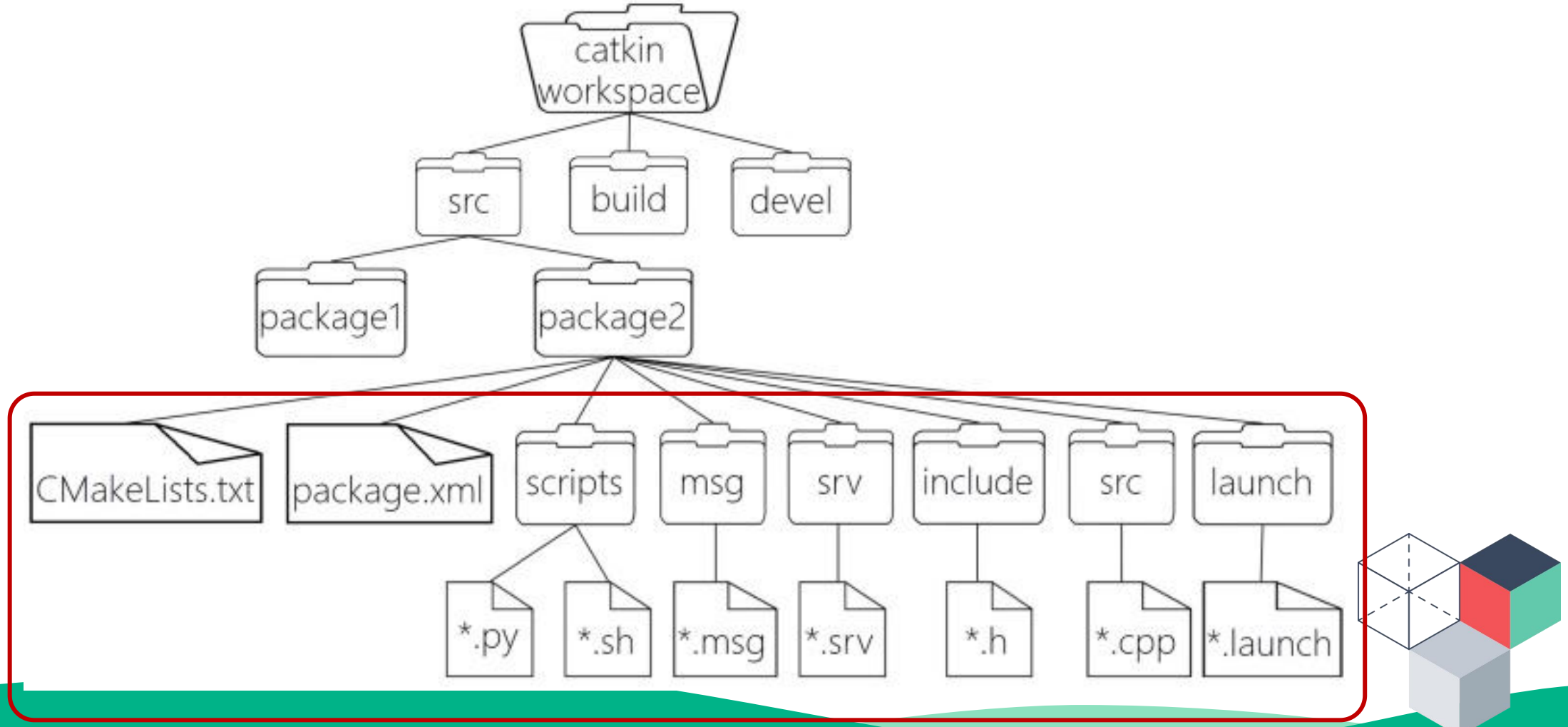
- catkin 工作空间中的功能包的结构：

```
workspace_folder/                   -- WORKSPACE
    src/                            -- SOURCE SPACE
        CMakeLists.txt              -- 'Toplevel' CMake file, provided by catkin
        package_1/
            CMakeLists.txt          -- CMakeLists.txt file for package_1
            package.xml             -- Package manifest for package_1
        ...
        package_n/
            CMakeLists.txt          -- CMakeLists.txt file for package_n
            package.xml             -- Package manifest for package_n
```



2.1 The filesystem level

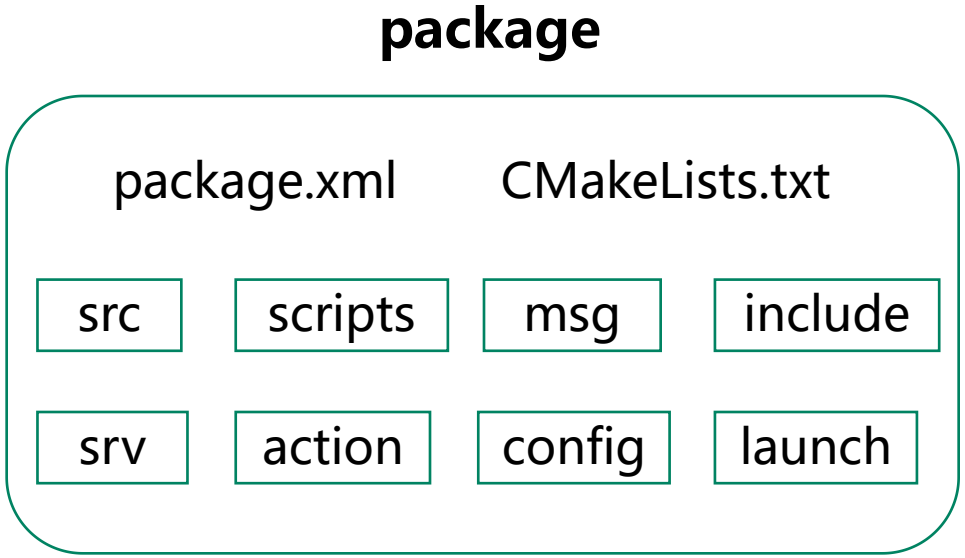
2.1.2 Catkin package structures

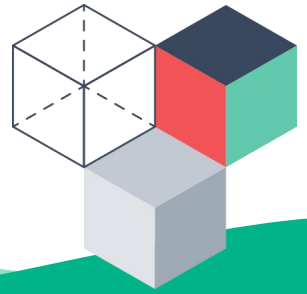


2.1 The filesystem level

2.1.2 Catkin package structures

- **功能包的一般结构如下所示：**
  - **config/: 保存了当前ROS包中使用的所有配置文件，由用户创建**
  - **include/package_name/: 此目录包含了所用库的头文件**
  - **msg/: 包含自定义的消息类型文件**
  - **scripts/: 包括 Bash、Pyhton 或其他脚本的可执行脚本文件**
  - **src/: 存储源文件的文件夹，例如为节点和nodelet创建的文件夹**
  - **srv/: 储存服务类型文件**
  - **action/: 储存动作类型文件**
  - **launch/: 保存一个或者多个ROS节点的启动文件**
  - **CMakeLists.txt: CMake的生成文件**
  - **package.xml: 功能包清单文件（必须放在功能包文件夹中，如果你发现某个文件夹中包含package.xml文件，那么这个文件夹很可能是功能包或者元功能包）**





2.1 The filesystem level

2.1.3 package.xml : http://wiki.ros.org/catkin/package.xml

1. 功能包清单：

- 它必须包含在任何符合 catkin 标准的包的根文件夹中
- 该文件定义了有关功能包的属性，例如包名称、版本号、作者、维护者以及对其他 catkin 包的依赖关系
- 该概念类似于旧版 rosbuild 构建系统中使用的 manifest.xml 文件
- 作用：package.xml 中声明功能包依赖项。在本机中，可以从源代码构建并在自己的机器上运行测试，但是将功能包在发布到 ROS 社区时将无法正常工作，其他人则依赖此信息来安装他们使用你的软件包所需的软件

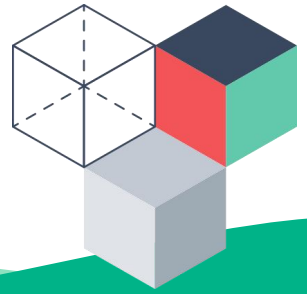
➢ 基本结构：每个 package.xml 文件都有 <package> 标记作为文档中的根元素

<package format="2">

<Name>元素</Name>

</package>

作业：学习 format1 和 format2 的用法和含义



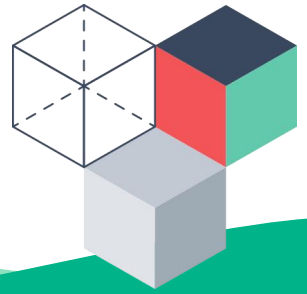
2.1 The filesystem level

2.1.3 package.xml

2. 必备标签：嵌套在 <package> 标签中最少的标签，以使包清单完整

- <name> - The name of the package
- <version> - The version number of the package (required to be 3 dot-separated integers)
- <description> - A description of the package contents
- <maintainer> - The name of the person(s) that is/are maintaining the package
- <license> - The software license(s) (e.g. GPL, BSD, ASL) under which the code is released

```
<package format="2">
  <name>foo_core</name>
  <version>1.2.4</version>
  <description>
  This package provides foo capability.
  </description>
  <maintainer email="ivana@osrf.org">Ivana Bildbotz</maintainer>
  <license>BSD</license>
</package>
```



2.1 The filesystem level

2.1.3 package.xml

- **依赖项：功能包可以有六种类型的依赖项**
  - **Build Dependencies specify which packages are needed to build this package. This is the case when any file from these packages is required at build time. This can be including headers from these packages at compilation time, linking against libraries from these packages or requiring any other resource at build time (especially when these packages are find_package()-ed in CMake). In a cross-compilation scenario build dependencies are for the targeted architecture.**
  - **Build Export Dependencies specify which packages are needed to build libraries against this package. This is the case when you transitively include their headers in public headers in this package (especially when these packages are declared as (CATKIN_)DEPENDS in catkin_package() in CMake).**
  - **Execution Dependencies specify which packages are needed to run code in this package. This is the case when you depend on shared libraries in this package (especially when these packages are declared as (CATKIN_)DEPENDS in catkin_package() in CMake).**

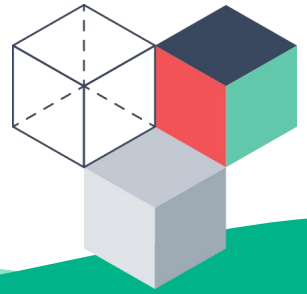


2.1 The filesystem level

2.1.3 package.xml

- **依赖项：功能包可以有六种类型的依赖项**

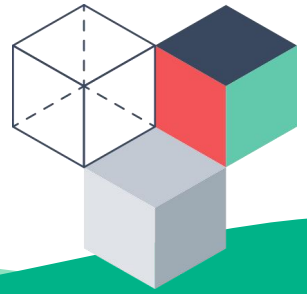
  - Test Dependencies specify only additional dependencies for unit tests. They should never duplicate any dependencies already mentioned as build or run dependencies.
  - Build Tool Dependencies specify build system tools which this package needs to build itself. Typically the only build tool needed is catkin. In a cross-compilation scenario build tool dependencies are for the architecture on which the compilation is performed.
  - Documentation Tool Dependencies specify documentation tools which this package needs to generate documentation.



2.1 The filesystem level

2.1.3 package.xml

- 依赖项标签：
  - <depend> specifies that a dependency is a build, export, and execution dependency. This is the most commonly used dependency tag.
  - <build_depend> → 编译依赖项，Build Dependencies
  - <build_export_depend> → 导出依赖项，Build Export Dependencies
  - <exec_depend> → 运行依赖项，Execution Dependencies
  - <test_depend> → 测试用例依赖项，Test Dependencies
  - <buildtool_depend> → 编译构建工具，Build Tool Dependencies
  - <doc_depend> → 文档依赖项，Documentation Tool Dependencies



2.1 The filesystem level

2.1.3 package.xml

- **A typical format2 package.xml:**

```
<package format="2">
  <name>foo_core</name>
  <version>1.2.4</version>
  <description>
    This package provides foo capability.
  </description>
  <maintainer email="ivana@willowgarage.com">Ivana Bildbotz</maintainer>
  <license>BSD</license>

  <url>http://ros.org/wiki/foo_core</url>
  <author>Ivana Bildbotz</author>

  <buildtool_depend>catkin</buildtool_depend>

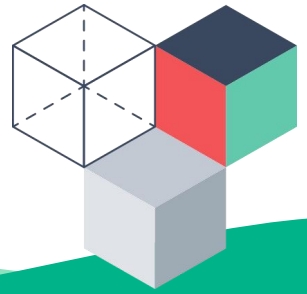
  <depend>roscpp</depend>
  <depend>std_msgs</depend>

  <build_depend>message_generation</build_depend>

  <exec_depend>message_runtime</exec_depend>
  <exec_depend>rospy</exec_depend>

  <test_depend>python-mock</test_depend>

  <doc_depend>doxygen</doc_depend>
</package>
```



2.1 The filesystem level

2.1.3 package.xml

- **Turtlesim package.xml**:

```
ros@ROS-VirtualBox:~$ roscd turtlesim
ros@ROS-VirtualBox:/opt/ros/melodic/share/turtlesim$ cat package.xml
<?xml version="1.0"?>
<package>
  <name>turtlesim</name>
  <version>0.9.3</version>
  <description>
    turtlesim is a tool made for teaching ROS and ROS packages.
  </description>
  <maintainer email="dthomas@osrfoundation.org">Dirk Thomas</maintainer>
  <license>BSD</license>

  <url type="website">http://www.ros.org/wiki/turtlesim</url>
  <url type="bugtracker">https://github.com/ros/ros_tutorials/issues</url>
  <url type="repository">https://github.com/ros/ros_tutorials</url>
  <author>Josh Faust</author>

  <buildtool_depend>catkin</buildtool_depend>

  <build_depend>geometry_msgs</build_depend>
  <build_depend>libboost-thread-dev</build_depend>
  <build_depend>qtbase5-dev</build_depend>
  <build_depend>message_generation</build_depend>
  <build_depend>qt5-qmake</build_depend>
  <build_depend>rosconsole</build_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>roscpp_serialization</build_depend>
  <build_depend>roslib</build_depend>
  <build_depend>rostime</build_depend>
  <build_depend>std_msgs</build_depend>
  <build_depend>std_srvs</build_depend>

  <run_depend>geometry_msgs</run_depend>
  <run_depend>libboost-thread-dev</run_depend>
  <run_depend>libqt5-core</run_depend>
  <run_depend>libqt5-gui</run_depend>
  <run_depend>message_runtime</run_depend>
  <run_depend>rosconsole</run_depend>
  <run_depend>roscpp</run_depend>
  <run_depend>roscpp_serialization</run_depend>
  <run_depend>roslib</run_depend>
  <run_depend>rostime</run_depend>
  <run_depend>std_msgs</run_depend>
  <run_depend>std_srvs</run_depend>
</package>
```



2.1.3 package.xml

3. 自定义功能包：

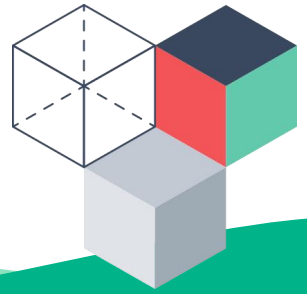
① 自定义 package.xml

- **描述标签**

```
切换行号显示

    5   <description>The beginner_tutorials package</description>
```

- **描述信息提供功能包的作用及使用时的注意事项等内容**
- **如果用一句话难以描述完全则需要换行**



2.1 The filesystem level

2.1.3 package.xml

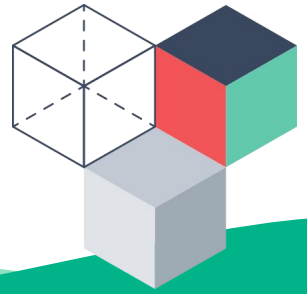
3. 自定义功能包：

- **维护者标签**

```
切换行号显示

 7   <!-- One maintainer tag required, multiple allowed, one person per tag -->
 8   <!-- Example:  -->
 9   <!-- <maintainer email="jane.doe@example.com">Jane Doe</maintainer> -->
10   <maintainer email="user@todo.todo">user</maintainer>
```

- **至少需要填写一个维护者，可以添加多个**
- **除了在标签里面填写维护者的名字外，还应该在标签的 email 属性中填写电子邮件地址**



2.1.3 package.xml

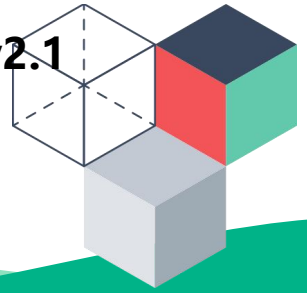
3. 自定义功能包：

- 许可证标签

```
切换行号显示

12   <!-- One license tag required, multiple allowed, one license per tag -->
13   <!-- Commonly used license strings: -->
14   <!--   BSD, MIT, Boost Software License, GPLv2, GPLv3, LGPLv2.1, LGPLv3 -->
15   <license>TODO</license>
```

- 选择一种许可证并将它填写到这里
- 常见的开源许可协议有 BSD、MIT、Boost Software License、GPLv2、GPLv3、LGPLv2.1和LGPLv3，可以在 Open Source Initiative 中阅读其中的若干个许可协议的相关信息



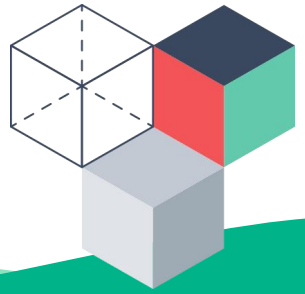
2.1.3 package.xml

3. 自定义功能包：

- **依赖项标签**

切换行号显示

```
27  <!-- The *_depend tags are used to specify dependencies -->
28  <!-- Dependencies can be catkin packages or system dependencies -->
29  <!-- Examples: -->
30  <!-- Use build_depend for packages you need at compile time: -->
31  <!--   <build_depend>genmsg</build_depend> -->
32  <!-- Use buildtool_depend for build tool packages: -->
33  <!--   <buildtool_depend>catkin</buildtool_depend> -->
34  <!-- Use exec_depend for packages you need at runtime: -->
35  <!--   <exec_depend>python-yaml</exec_depend> -->
36  <!-- Use test_depend for packages you need only for testing: -->
37  <!--   <test_depend>gtest</test_depend> -->
38  <buildtool_depend>catkin</buildtool_depend>
39  <build_depend>roscpp</build_depend>
40  <build_depend>rospy</build_depend>
41  <build_depend>std_msgs</build_depend>
```



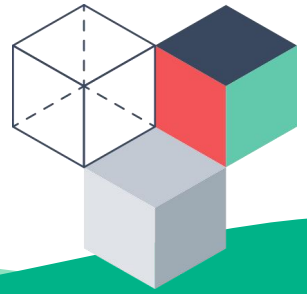
2.1.3 package.xml

3. 自定义功能包：

- 依赖项标签

```
38   <buildtool_depend>catkin</buildtool_depend>
39   <build_depend>roscpp</build_depend>
40   <build_depend>rospy</build_depend>
41   <build_depend>std_msgs</build_depend>
```

- 此标签描述了功能包的依赖关系，这些依赖项分为 build_depend、buildtool_depend、exec_depend、test_depend
- 创建功能包时将 roscpp、rospy 和 std_msgs 作为 catkin_create_pkg 命令的参数



2.1.3 package.xml

3. 自定义功能包：

- 依赖项标签

切换行号显示

```
12  <buildtool_depend>catkin</buildtool_depend>
13
14  <build_depend>roscpp</build_depend>
15  <build_depend>rospy</build_depend>
16  <build_depend>std_msgs</build_depend>
17
18  <exec_depend>roscpp</exec_depend>
19  <exec_depend>rospy</exec_depend>
20  <exec_depend>std_msgs</exec_depend>
```

- 在本例中，因为在编译和运行时都需要用到所有指定的依赖包，因此将每一个依赖包分别添加到 run_depend 标签中



2.1 The filesystem level

2.1.3 package.xml

切换行号显示

```
<?xml version="1.0"?>
<package format="2">
  <name>beginner_tutorials</name>
  <version>0.1.0</version>
  <description>The beginner_tutorials package</description>

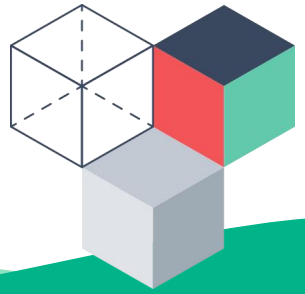
  <maintainer email="you@yourdomain.tld">Your Name</maintainer>
  <license>BSD</license>
  <url type="website">http://wiki.ros.org/beginner_tutorials</url>
  <author email="you@yourdomain.tld">Jane Doe</author>

  <buildtool_depend>catkin</buildtool_depend>

  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>

  <exec_depend>roscpp</exec_depend>
  <exec_depend>rospy</exec_depend>
  <exec_depend>std_msgs</exec_depend>

</package>
```



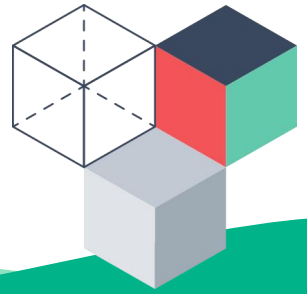
完整的package.xml文件

2.1 The filesystem level

2.1.4 CMakeList.txt：http://wiki.ros.org/catkin/CMakeLists.txt

1. Overview：

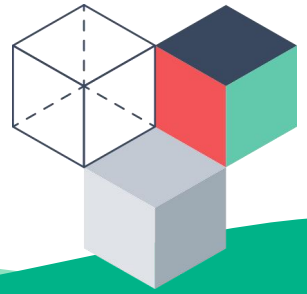
- CMakeLists.txt 是用于构建软件包的 CMake 构建系统的输入
- 任何符合 CMake 的功能包都包含一个或多个 CMakeLists.txt 文件，这些文件描述了如何构建代码以及将代码安装到何处
- 用于 catkin 项目的 CMakeLists.txt 文件是带有一些附加约束的标准 vanilla CMakeLists.txt 文件



2.1.4 CMakeList.txt

2. Overall Structure and Ordering：作业：自行学习，下节课先测试，再讲解

- Required CMake Version (cmake_minimum_required)
- Package Name (project())
- Find other CMake/Catkin packages needed for build (find_package())
- Enable Python module support (catkin_python_setup())
- Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())
- Invoke message/service/action generation (generate_messages())
- Specify package build info export (catkin_package())
- Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())
- Tests to build (catkin_add_gtest())
- Install rules (install())



2.1 The filesystem level

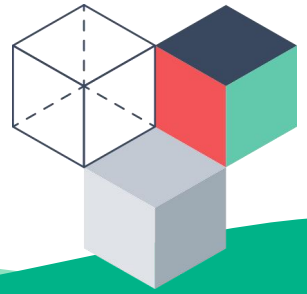
2.1.5 Metapackages

- **元功能包**：将多个功能包分组为单个逻辑包。 元功能包是一个普通功能包，在 package.xml 中具有以下导出标记：

```
<export>
    <metapackage />
</export>
```

  - 除了必需的 <buildtool_depends> 对 catkin 的依赖之外，元功能包只能对它们分组的包具有执行依赖关系
  - 元功能包有一个必需的样板 CMakeLists.txt 文件：

```
cmake_minimum_required(VERSION 2.8.3)
project(<PACKAGE_NAME>)
find_package(catkin REQUIRED)
catkin_metapackage()
```



Post-assessment

1. XML 文档声明包括哪些内容?
2. XML 文档元素的特征是?
3. Catkin 功能包的最小结构?
4. Package.xml 和 CMakelists.txt 分别有什么作用?
5. Catkin 功能包中 src、srv、msg、launch 各存放什么文件?
6. Package.xml 的6个依赖项标签分别是?

主观题 10分

设置

1. XML 文档声明包括哪些内容?
2. XML 文档元素的特征是?
3. Catkin 功能包的最小结构?
4. Package.xml 和 CMakelists.txt 分别有什么作用?
5. Catkin 功能包中 src、srv、msg、launch 各存放什么文件?
6. Package.xml 的 6个依赖项标签分别是?

正常使用主观题需2.0以上版本雨课堂

作答

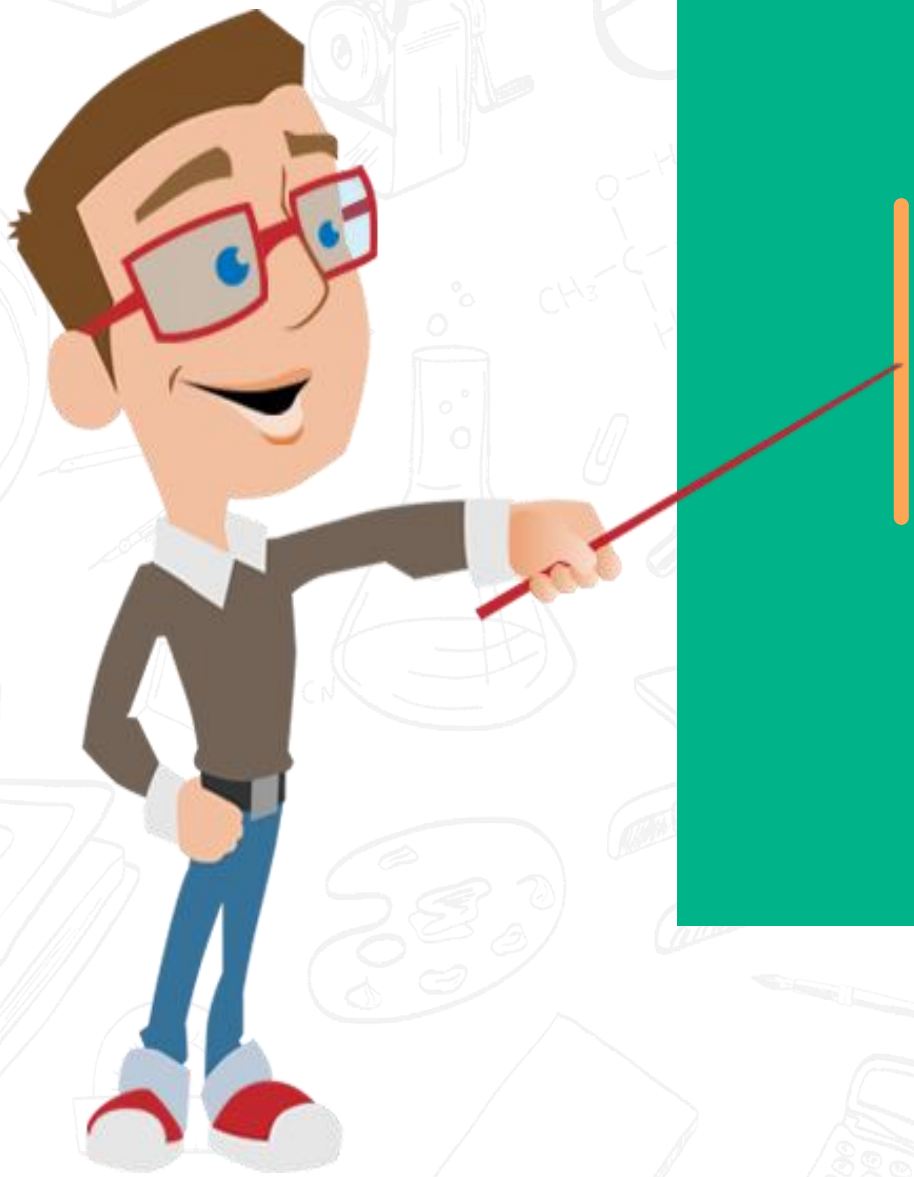
Summary

1. XML文件：文件结构（文档声明、元素、属性、注释）
2. ROS文件系统
    a) 文件系统架构 
    b) 功能包架构
    c) rosbash
    d) catkin功能包 
3. package.xml： 作用、基本结构、必备标签、依赖项 

Q&A

今日事今日毕

Question?



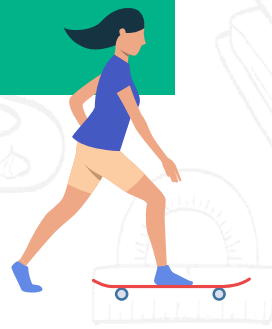
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 6-3

1. Message and Service
2. ~
3. ~
4. ~

Objectives

1、掌握 ROS Message 的定义和使用方法

2、掌握 ROS Service 的定义和使用方法

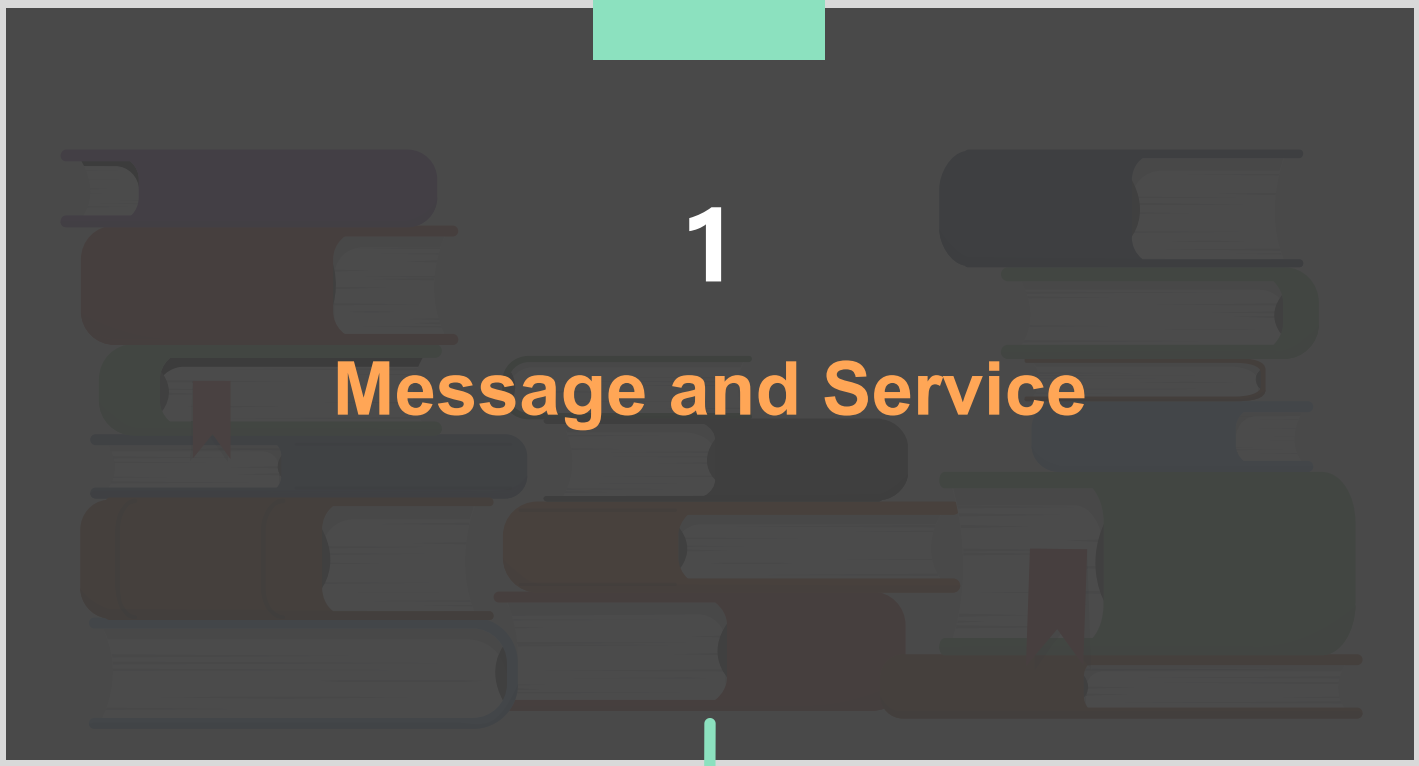


Pre-assessment

1、是否了解和使用过 ROS Message?

2、是否了解和使用过 ROS Service?

3、了解 ROS IDE?



1

Message and Service

1.1 ROS Message

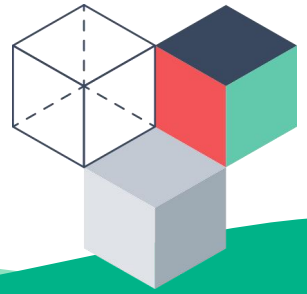
1.1.1 Overview： http://wiki.ros.org/msg

- ROS 使用简化的 消息描述语言 来描述 ROS 消息类型（也称为消息）
- ROS消息的作用是描述节点发布的数据值
- 消息描述存储在 ROS 软件包的 msg 文件夹下，以 .msg 为扩展名
- 消息类型有2种方式：字段(Fields) 和 常量(Constants)
  - 字段：定义了在消息中传输的数据类型，包含字段类型(Field Type)和字段名称(Field Name)2部分，如 int32 x、float32 y、string s 等
  - 常量：定义了可用于解释这些字段的值
- 每个字段由一个字段类型和一个字段名称组成，用空格分隔，其语法格式为：

```
fieldtype1 fieldname1
fieldtype2 fieldname2
```

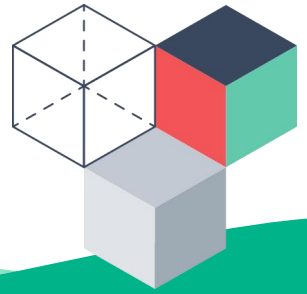
- 举例：

```
int32 x
int32 y
```



1.1.2 Fields——Field Type

- 字段类型可以是以下四种之一：
  a) 内置类型，例如“float32 pan”或“string name”
  b) 自定义的消息描述的名称，例如“geometry_msgs/PoseStamped”
  c) 上述的固定或可变长度数组（列表），例如“float32[] ranges”或“Point32[10] points”
  d) 特殊的 Header 类型，该类型映射到 std_msgs/Header
- 嵌入其他消息描述时，如果在同一个功能包中，类型名称可能是相对的（例如“Point32”）；否则它必须是完整的消息类型（例如“std_msgs/String”）。 此规则的唯一例外是 Header
- 注意：在构建自己的消息类型时，不得使用内置类型或 Header 的名称

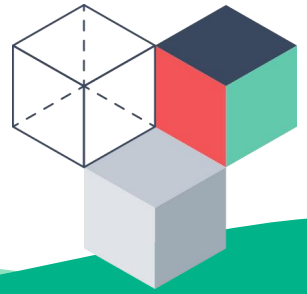


1.1 ROS Message

1.1.2 Fields——Field Type

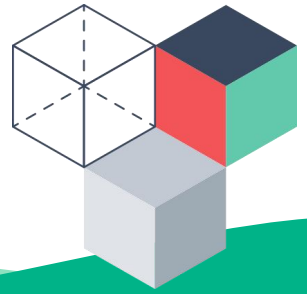
常用字段类型：字段类型与 C++ 和 Python 中的对应关系

Primitive Type	Serialization	C++	Python2 / Python3
bool	unsigned 8-bit int	uint8_t	bool
int8	signed 8-bit int	int8_t	int
uint8	unsigned 8-bit int	uint8_t	int
int16	signed 16-bit int	int16_t	int
uint16	unsigned 16-bit int	uint16_t	int
int32	signed 32-bit int	int32_t	int
uint32	unsigned 32-bit int	uint32_t	int
int64	signed 64-bit int	int64_t	long int
uint64	unsigned 64-bit int	uint64_t	long int
float32	32-bit IEEE float	float	float
float64	64-bit IEEE float	double	float
string	ascii string	std::string	str bytes
time	secs/nsecs unsigned 32-bit ints	ros::Time	rospy.Time
duration	secs/nsecs signed 32-bit ints	ros::Duration	rospy.Duration



1.1.2 Fields——Field Name

- **字段名称**决定了如何在目标语言中引用数据值
  - 例如，一个名为“pan”的字段名称在 Python 中将被引用为“obj.pan”，假设“obj”是存储消息的变量
- **字段名称**需要由消息生成器翻译成多种目标语言，因此
  - 将字段名称限制为字母字符后跟字母数字和下划线的任意混合，即 [a-zA-Z][a-zA-Z1-9_]*
  - **避免**使用与常用语言中的**关键字**相对应的字段名称



1.1 ROS Message

1.1.3 Constants

- 常量定义与字段描述类似，区别是常量定义会分配一个值，此值分配通过使用等号“=”表示

  constanttype1 CONSTANTNAME1=constantvalue1

  constanttype2 CONSTANTNAME2=constantvalue2

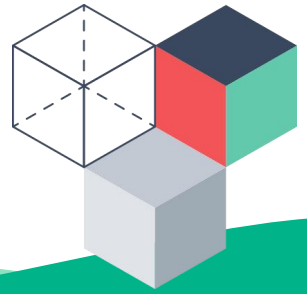
- 举例：

  int32 X=123

  int32 Y=-123

  string FOO=“foo”

- 注意：
  - 除了time和duration，可以将任何内置类型声明为常量
  - string 常量被赋予等号右侧所有内容的值，去除前导和尾随空格。不能对 string 常量添加注释
  - 整数常量必须以十进制（基数为 10）指定

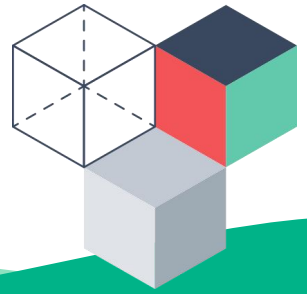


1.1 ROS Message

1.1.4 Common_msgs

- 除了内置类型，ROS已经将常用到的一些消息。如传感器消息、导航消息等设置成了不同的消息类型；common_msgs 是各种常用的消息包的集合，主要包括：

1. actionlib_msgs
2. diagnostic_msgs
3. geometry_msgs → rosmsg show geometry_msgs/TAB
4. nav_msgs
5. sensor_msgs → 参考：https://blog.csdn.net/qq_36764147/article/details/103100498
6. shape_msgs
7. stereo_msgs
8. trajectory_msgs
9. visualization_msgs



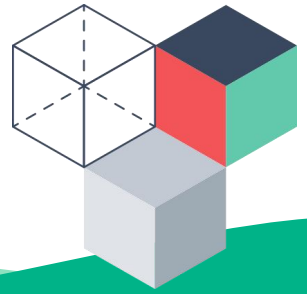
1.1 ROS Message

1.1.5 Geometry_msgs：https://wiki.ros.org/geometry_msgs?distro=melodic

- geometry_msgs 为常见的几何基元（例如点、向量和姿势）提供消息。 这些几何基元旨在提供通用数据类型并促进整个系统的互操作性

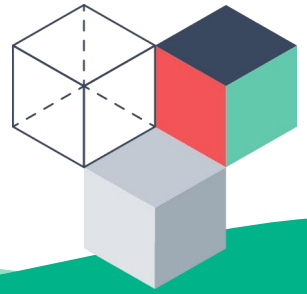
- geometry_msgs message types:

Accel
AccelStamped
AccelWithCovariance
AccelWithCovarianceStamped
Inertia
InertiaStamped
Point
Point32
PointStamped
Polygon
PolygonStamped
Pose
Pose2D
PoseArray
PoseStamped
PoseWithCovariance
PoseWithCovarianceStamped
Quaternion
QuaternionStamped
Transform
TransformStamped
Twist
TwistStamped
TwistWithCovariance
TwistWithCovarianceStamped
Vector3
Vector3Stamped
Wrench
WrenchStamped



1.1.6 Building .msg Files

- **自定义消息类型**:
  - ROS 客户端库通过消息生成器（message generators）将 .msg 文件转换为源代码
  - 对于自定义.msg文件，需要确保 CMake 在你定义.msg文件后重新配置项目
  - 消息生成器必须从所构建脚本中调用，因此需要修改 package.xml 文件
  - 所有 .msg 文件存储在 msg 的目录中，需要编辑 CMakeLists.txt 文件来构建这些内容



1.1 ROS Message

1.1.6 Building .msg Files

- **自定义消息类型：**

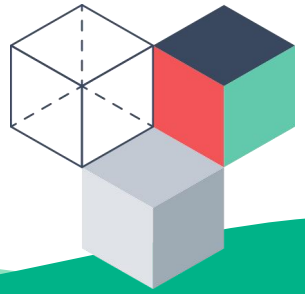
1. 添加 .msg 文件：在 Package 中添加 msg 文件夹，且添加自定义的 test.msg 文件

```
int32 x
int32 y
```

2. 修改 package.xml 文件：打开 package.xml，向其中添加以下两行代码：

```
<build_depend>message_generation</build_depend>
<run_depend>message_runtime</run_depend>
```

```
<buildtool_depend>catkin</buildtool_depend>
<build_depend>roscpp</build_depend>
<build_depend>rospy</build_depend>
<build_depend>std_msgs</build_depend>
<build_depend>message_generation</build_depend>
<build_export_depend>roscpp</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>
<exec_depend>roscpp</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>std_msgs</exec_depend>
<exec_depend>message_runtime</exec_depend>
```



1.1.6 Building .msg Files

- **自定义消息类型：**

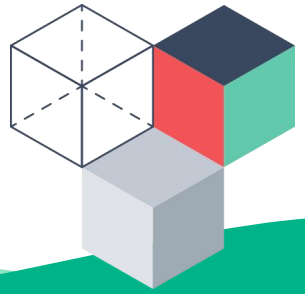
3. **修改 CMakeLists.txt 文件：find_package 和 catkin_package**

修改 find_package：将 message_generation 依赖项添加到 find_package，以便可以生成消息

```
## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

修改catkin_package：

```
catkin_package(
#  INCLUDE_DIRS include
#  LIBRARIES beginner_tutorials
#  CATKIN_DEPENDS roscpp rospy std_msgs
#  DEPENDS system_lib
   CATKIN_DEPENDS message_runtime
)
```



1.1.6 Building .msg Files

- **自定义消息类型：**

4. **修改 CMakeLists.txt 文件：add_message_files 和 generate_messages**

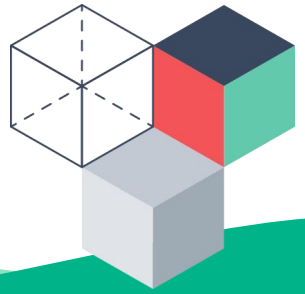
修改 add_message_files：

```
## Generate messages in the 'msg' folder
add_message_files(
  FILES
#   Message1.msg
#   Message2.msg
    test.msg
)
```

修改 generate_messages：

```
## Generate added messages and services with any dependencies listed here
generate_messages(
    DEPENDENCIES
    std_msgs
# )
```

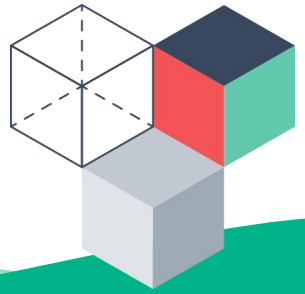
通过自定义 .msg 文件，确保 CMake 在添加其他 .msg 文件后知道如何重新配置项目



1.1 ROS Message

1.1.7 Command-line tools

- 适用于ROS消息的命令工具：rosmsg (ros+message)，用于查看消息类型数据结构的定义
  - rosmsg show：Show message description
  - rosmsg info：Alias for rosmsg show
  - rosmsg list：List all messages
  - rosmsg md5：Display message md5sum
  - rosmsg package：List messages in a package
  - rosmsg packages：List packages that contain messages



1.1 ROS Message

1.1.7 Command-line tools

- **举例说明：**

List messages in a package

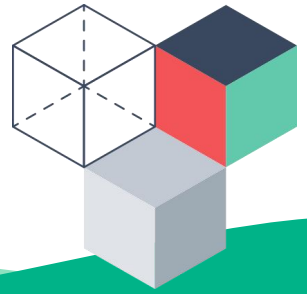
```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials$ rosmsg package beginner_t
utorials
beginner_tutorials/test
```

Show message description

```
ros@ROS-VirtualBox:~$ rosmsg show test
[beginner_tutorials/test]:
int32 x
int32 y
```

Show message description

```
ros@ROS-VirtualBox:~$ rosmsg info test
[beginner_tutorials/test]:
int32 x
int32 y
```

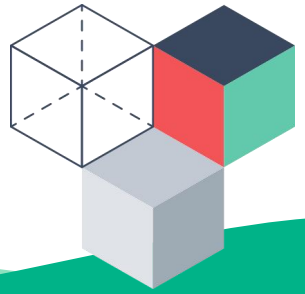


1.2 ROS Service

1.2.1 Overview

- ROS 使用简化的 服务描述语言 来描述 ROS 服务类型
- 服务的描述存储在软件包的 srv 目录下的 .srv 文件中
- ROS服务的作用是实现节点之间的 请求/响应 通信
- 若要调用服务，需要使用 软件包的名称 和 服务名称
  - 举例1： 对于sample_package/srv/sample.srv文件，可将其称为 sample_package/sample 服务（消息类型同样可以使用这种简化方法）
  - 举例2：

```
$ roscervice call /spawn 3 3 0 turtle2
```



1.2.2 Service Description Specification

- 服务描述文件由 请求 和 响应 消息类型组成，以“---”分隔。 用“---”连接在一起的任何两个 .msg 文件都是合法的服务描述
- 一个简单的服务示例，其接受一个字符串，并返回一个字符串：

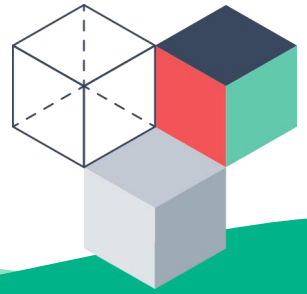
  string str

  ---

  string str

  备注：string 同 ROS message 中的字段类型

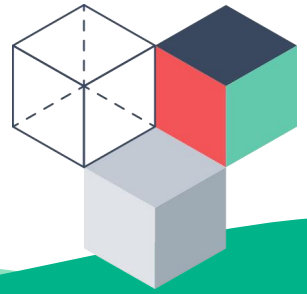
- 不能在 .srv 中嵌入另一个 .srv



1.2.2 Service Description Specification

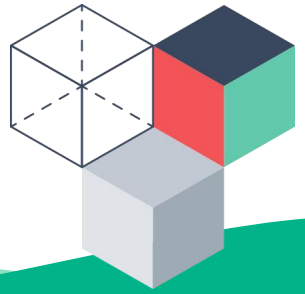
- 一个复杂的服务示例：

```
# request constants
int8 FOO=1
int8 BAR=2
# request fields
int8 foobar
another_pkg/AnotherMessage msg
---
# response constants
uint32 SECRET=123456
# response fields
another_pkg/YetAnotherMessage val
CustomMessageDefinedInThisPackage value
uint32 an_integer
```



1.2.3 Command-line tools

- rossrv 命令行工具显示有关 ROS 服务的信息。 它的用法与 rosmsg 完全相同
  - rossrv show：Show service description
  - rossrv list：List all services
  - rossrv md5：Display service md5sum
  - rossrv package：List services in a package
  - rossrv packages：List packages that contain services



1.2 ROS Service

1.2.3 Command-line tools

- **举例说明：**

List services in a package

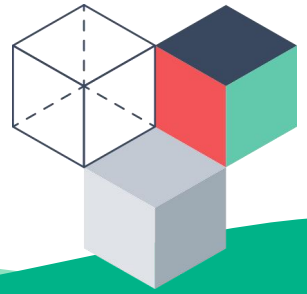
```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials/srv$ rossrv package beginner_tutorials
beginner_tutorials/test
```

Show service description

```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials/srv$ rossrv show test
[beginner_tutorials/test]:
string str
---
string str
```

Show service description

```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials/srv$ rossrv info test
[beginner_tutorials/test]:
string str
---
string str
```



Post-assessment

1. ROS 消息包括哪2种类型?
2. ROS 消息字段包括哪2部分?
3. 字段和常量的区别是?
4. ROS 消息和服务的作用分别是?
5. ROS 服务有哪2部分组成?

主观题 10分

设置

1. ROS 消息包括哪2种类型?
2. ROS 消息字段包括哪2部分?
3. 字段和常量的区别是?
4. ROS 消息和服务的作用分别是?
5. ROS 服务有哪2部分组成?

正常使用主观题需2.0以上版本雨课堂

作答

Summary

1. Message and Service

a) ROS Message:

① fields and constants

② command line tools

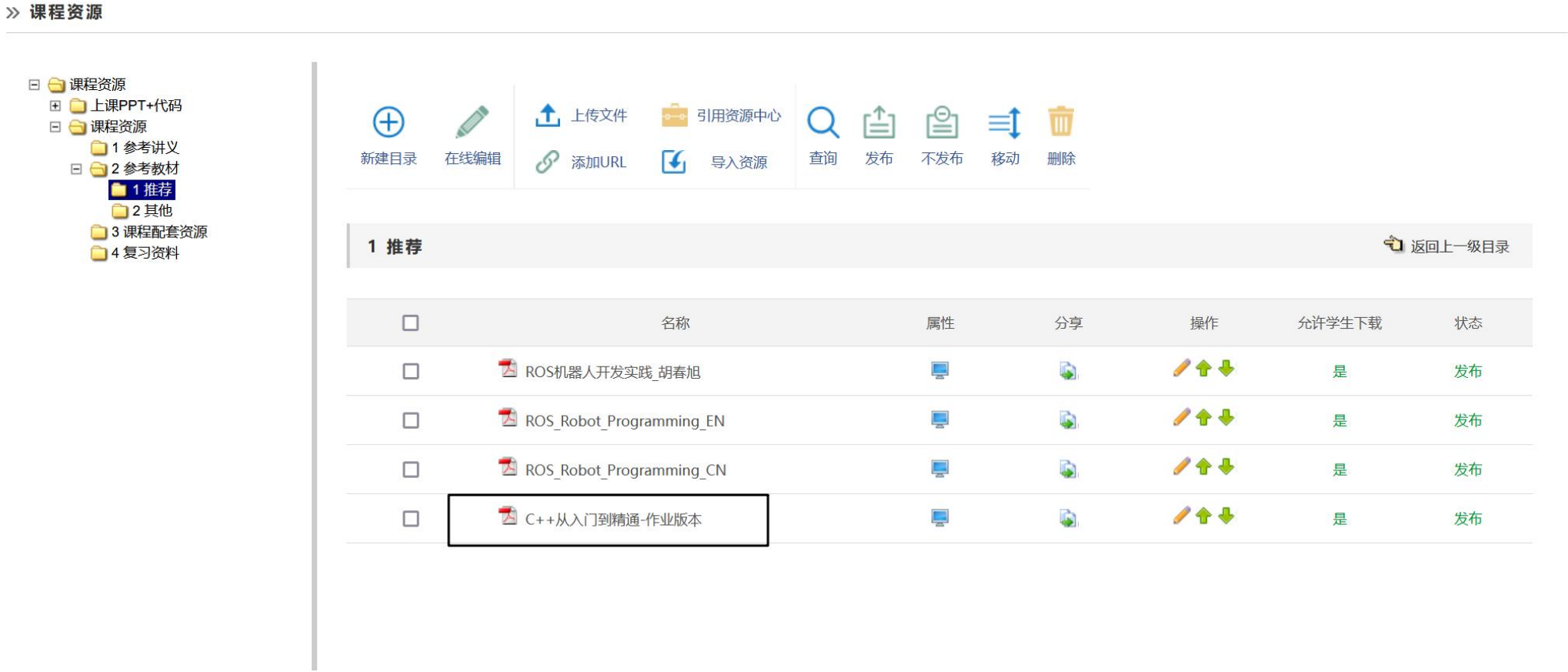
③ custom message type

b) ROS Service:

① description specification

② command line tools

Homework



作业：学习《C++从入门到精通-作业版本》第7章

Q&A

今日事今日毕

Question?